AgriScan System Documentation

Welcome to the **AgriScan** documentation. This document provides a comprehensive overview of the platform's architecture, security, features, and technical implementation.

🚀 Project Overview

AgriScan is an AI-powered agricultural platform designed to empower small-scale farmers with advanced diagnostic tools, real-time insights, and a connected community. Its mission is to bridge the gap between technology and traditional farming through accessible AI.

Core Features:

1. **AI Crop Disease Scanner**: High-precision image analysis to detect diseases in various crops (Maize, Wheat, Tomato, etc.).
2. **RAG-powered AI Assistant**: A real-time chat interface providing agricultural advice backed by a localized knowledge base.
3. **Digital Marketplace**: Connecting farmers with verified product buyers and agrovet agencies.
4. **Farmer Profile Dashboard**: A centralized hub for managing personal data, tracking registered crops, and viewing local weather forecasts.
5. **Multi-Language UI**: Support for Dark/Light modes and responsive design for all devices.

🛠 Tech Stack

Core Technologies:

- **Framework**: Django 5.x (Python)
- **AI Models**: TensorFlow / Keras (EfficientNet-based models)
- **LLM/Chat**: Groq API (Llama 3.1 8B/70B)
- **Vector Database**: ChromaDB (for RAG context)
- **Embeddings**: HuggingFace (`all-MiniLM-L6-v2`)

Frontend:

- **Styling**: Vanilla CSS3 with CSS Variables (Theming)
- **Logic**: Vanilla JavaScript (AJAX, DOM manipulation)
- **Icons**: FontAwesome 6

🔒 Security Architecture

AgriScan is built with a "Security-First" approach:

- **Session Security**: 30-minute inactivity timeout enforced via `SESSION_COOKIE_AGE`.
- **Authentication**: Secure login/signup system with Google OAuth2 integration.

- **Input Sanitization**: All user-provided data is sanitized via `strip_tags()` and regex validation to prevent SQLi and XSS.
- **File Security**:
  - Uploaded images are renamed to **UUIDs** to prevent path traversal.
  - MIME-type white-listing (JPEG, PNG, WEBP).
  - 5MB file size limit enforced.
- **XSS Protection**: Frontend rendering uses `.textContent` instead of `.innerHTML` to prevent script injection.

🏗️ System Architecture

1. **Inference Pipeline**: User uploads an image → Backend validates size/type → Image is processed/renamed → TensorFlow model predicts disease → Results returned with confidence score.
2. **RAG Assistant Pipeline**: User message → ChromaDB retrieval (top 3 relevant docs) → Prompt engineering with context → Groq API completion → AI response streamed/rendered safely.
3. **Directory System**: Django ORM-based filtering of Agencies and Markets with real-time county-based weather integration.

💻 Setup & Installation

1. Prerequisites:

- Python 3.10+
- A Groq API Key
- A Google Cloud OAuth client (optional, for Google Sign-in)

2. Local Environment Setup:

```
# Clone the repository
git clone <repo_url>
cd crop_AI_analyzer

# Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt

# Run migrations
python manage.py migrate

# Start the server
python manage.py runserver
```

3. Environment Variables (`.env`):

Create a `.env` file in the project root with:

```
GROQ_API_KEY=your_key_here
HF_TOKEN=your_huggingface_token
GOOGLE_CLIENT_ID=your_id_here
GOOGLE_CLIENT_SECRET=your_secret_here
SECRET_KEY=your_django_secret
DEBUG=True
```

🗺️ Roadmap & Limitations

Known Limitations:

- AI Scanner requires clear, well-lit leaf photos for optimal accuracy.
- Weather data depends on the accuracy of the user's registered county.

Future Roadmap:

- [ ] **Offline Mode**: Lite model for areas with low connectivity (TFLite).
- [ ] **Push Notifications**: Real-time alerts for weather changes or pest outbreaks.
- [ ] **Advanced Analytics**: Seasonal crop yield predictions based on historical profile data.

© 2026 AgriScan Development Team.